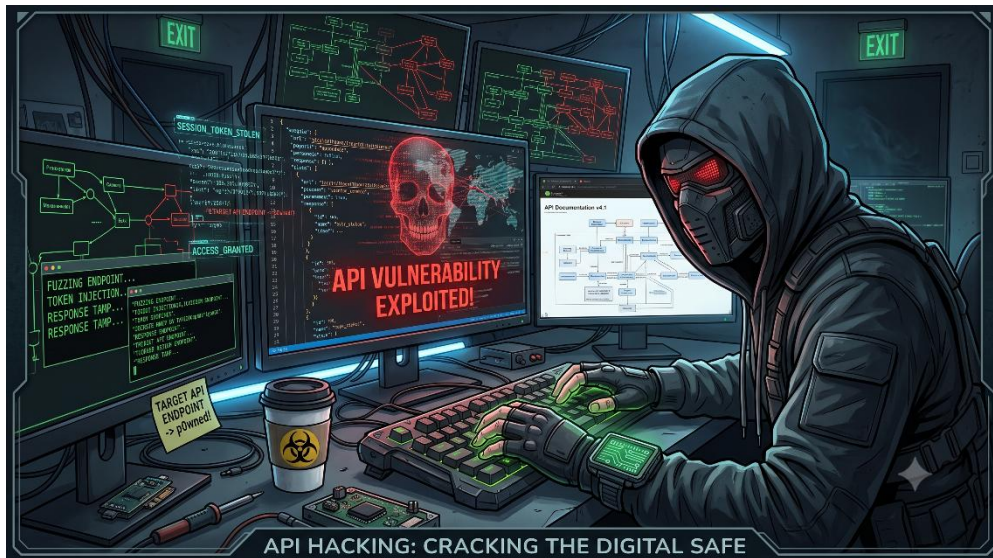


Secure shop Api Security Lab

@Aditya Bhosale

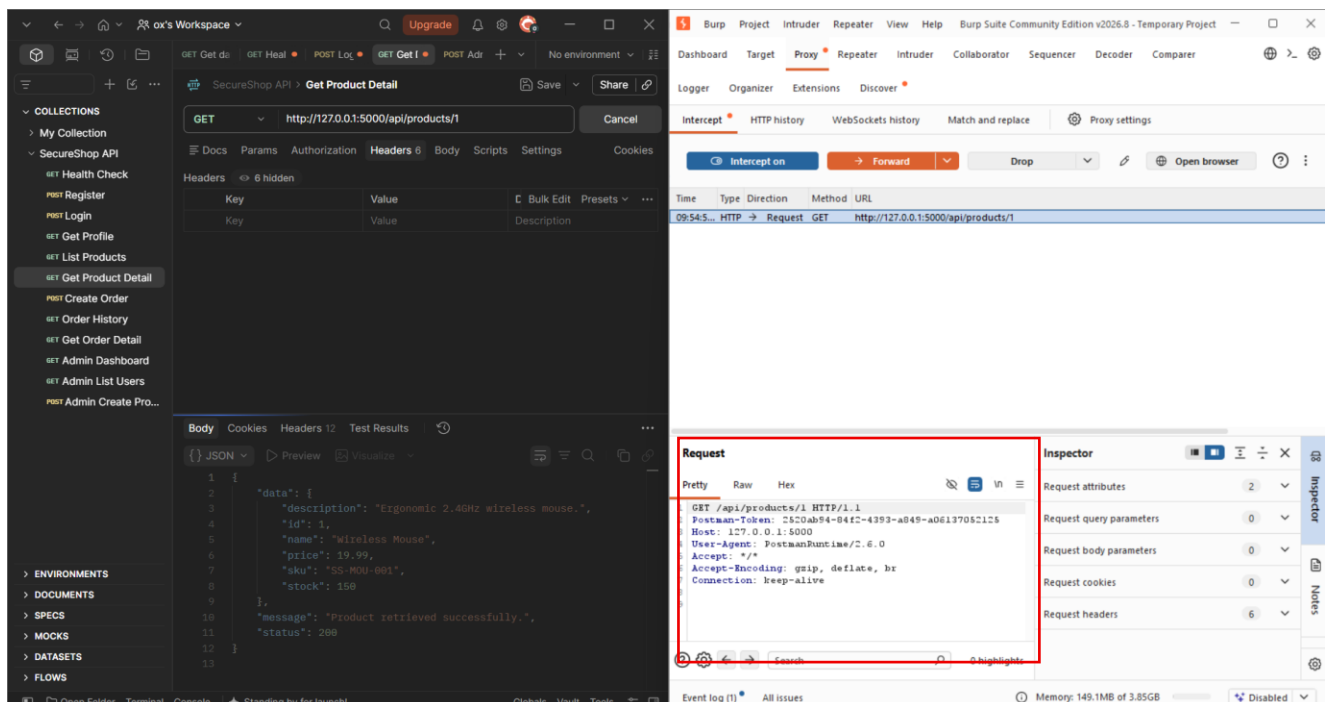
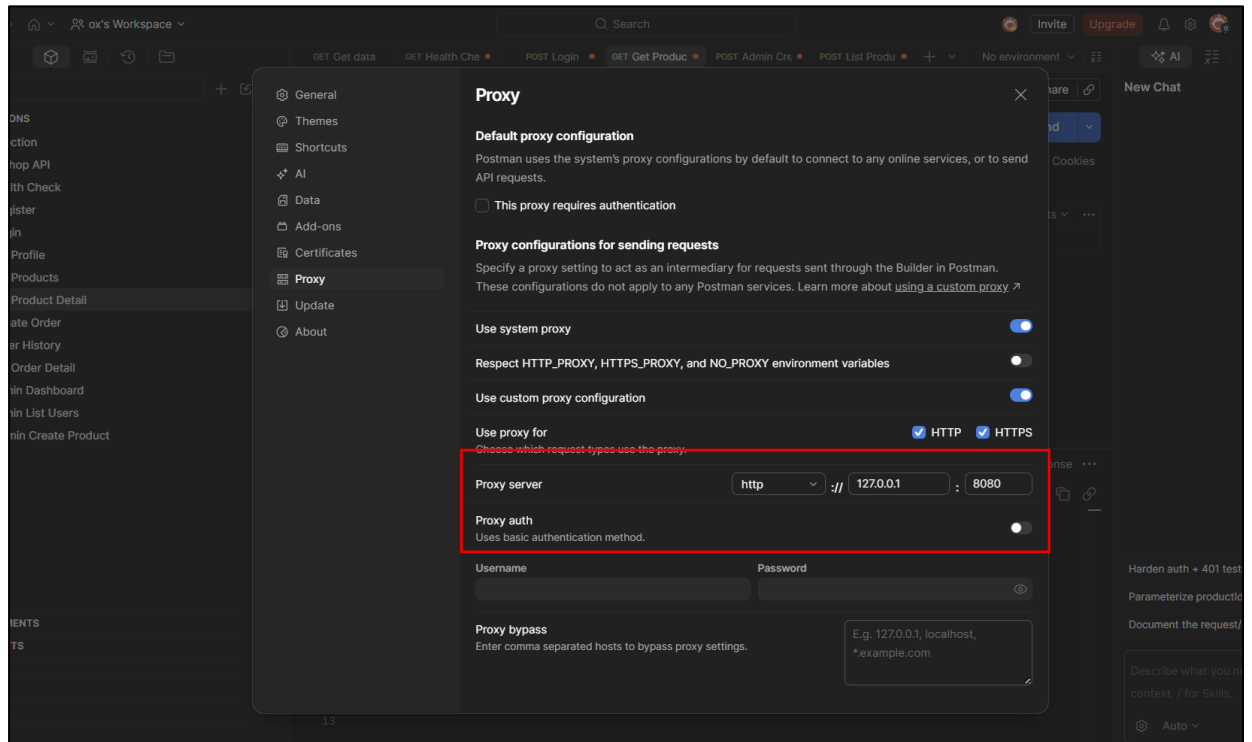


SecureShop — A Flask/JWT REST API with production-grade security controls and a parallel vulnerable mode mapped to the OWASP API Top 10.

This Document an API security testing guide covering Burp Suite and Postman configuration, testing methodologies, and security assessment checklists. Manually tested the SecureShop API in both secure and vulnerable modes to identify and assess API security vulnerabilities.

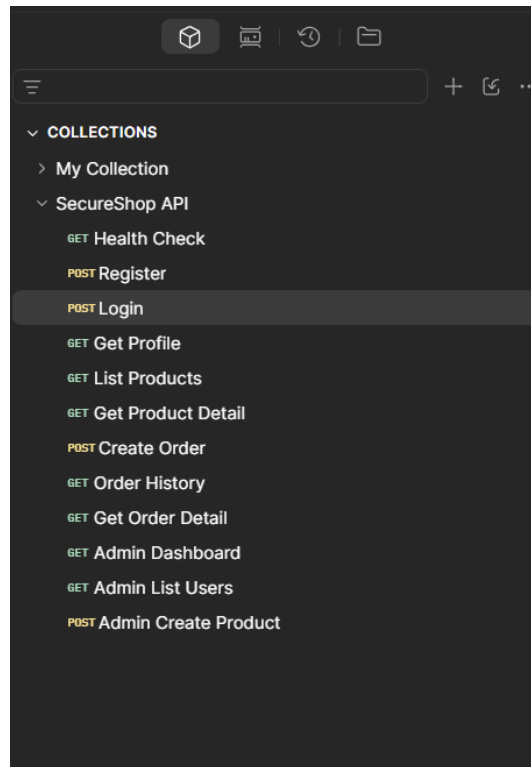
Setup API Testing Lab Postman + Burp suite

- Open Postman → Import → select the collection and environment files from postman/. Skip this if using a dynamic API.
- In Postman Settings, match the proxy with Burp Suite. You're ready to go.

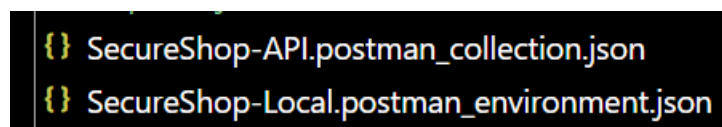


Now we can see the requests in Burp Suite. Our playground is ready.

List of APIs used in project:

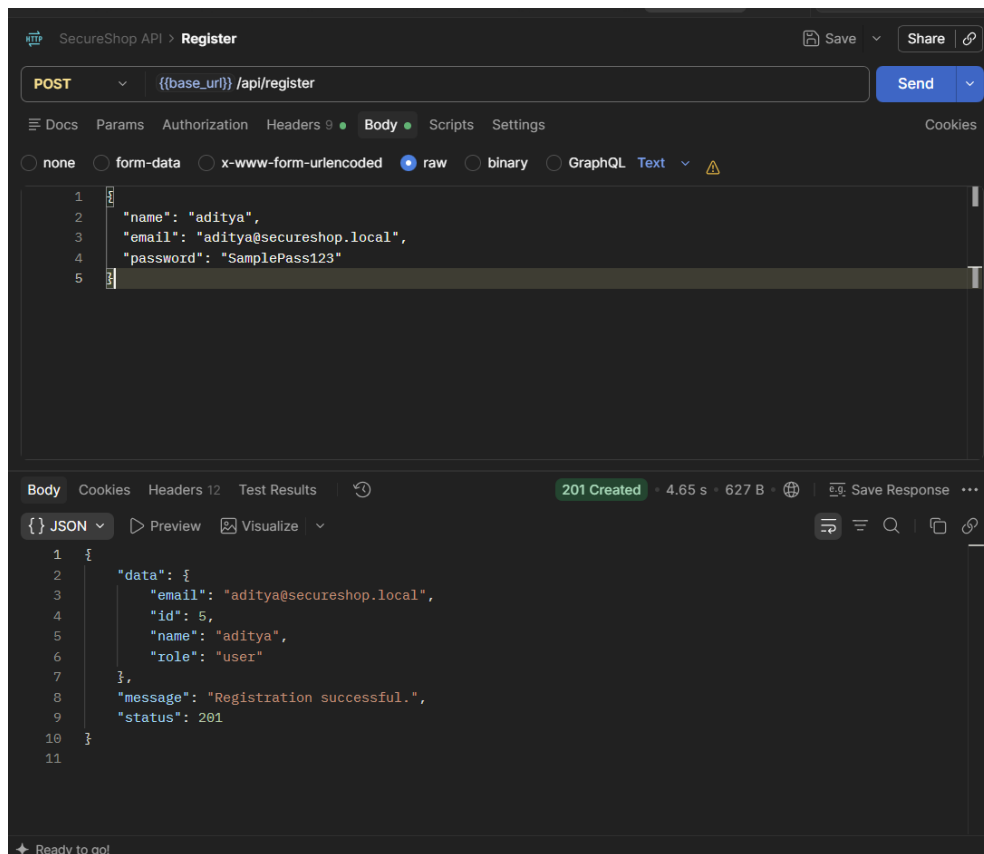


Inside postman folder those file you have to import in postman:

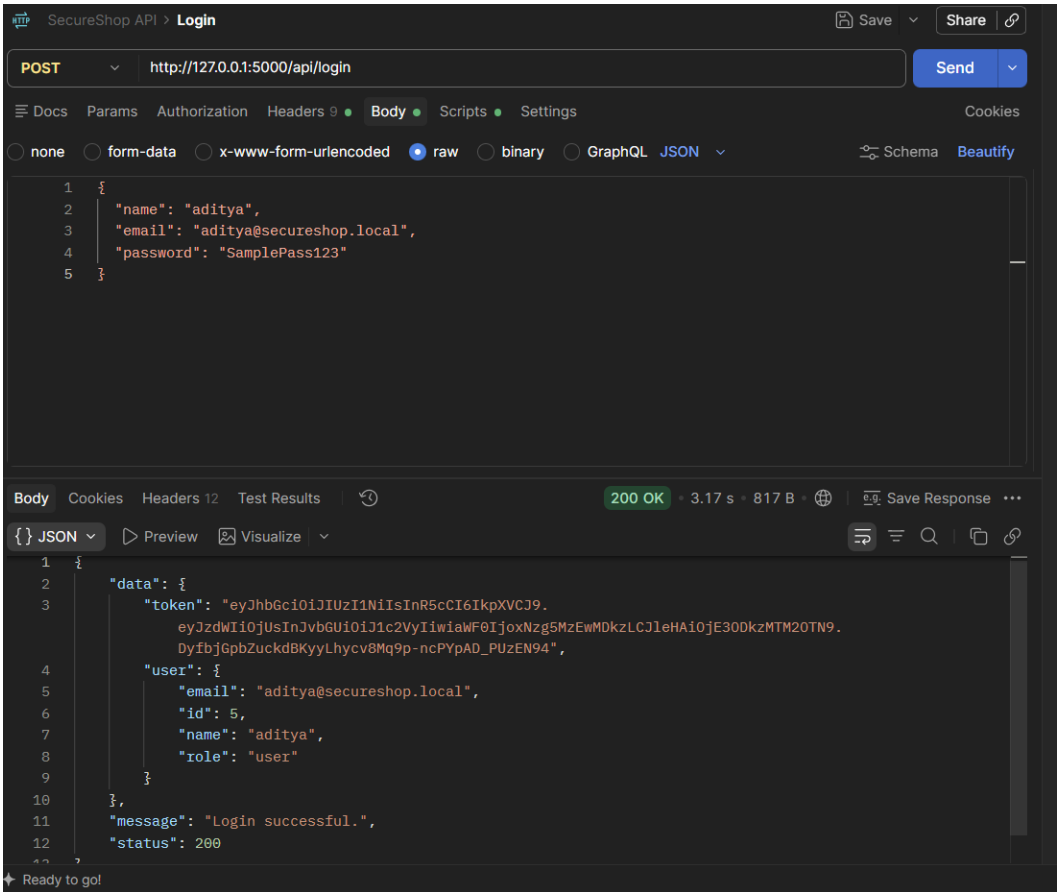


The test cases and event tested are with and without training mode you can do it on your own its good for someone who is new into api and security testing.

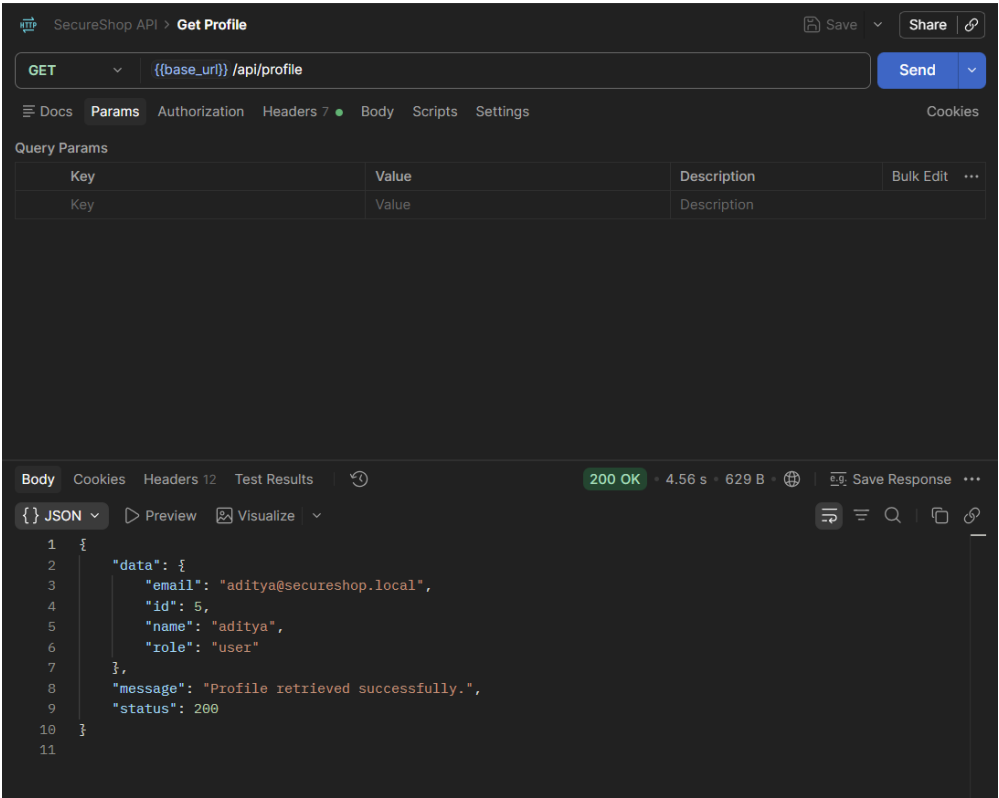
Registering user



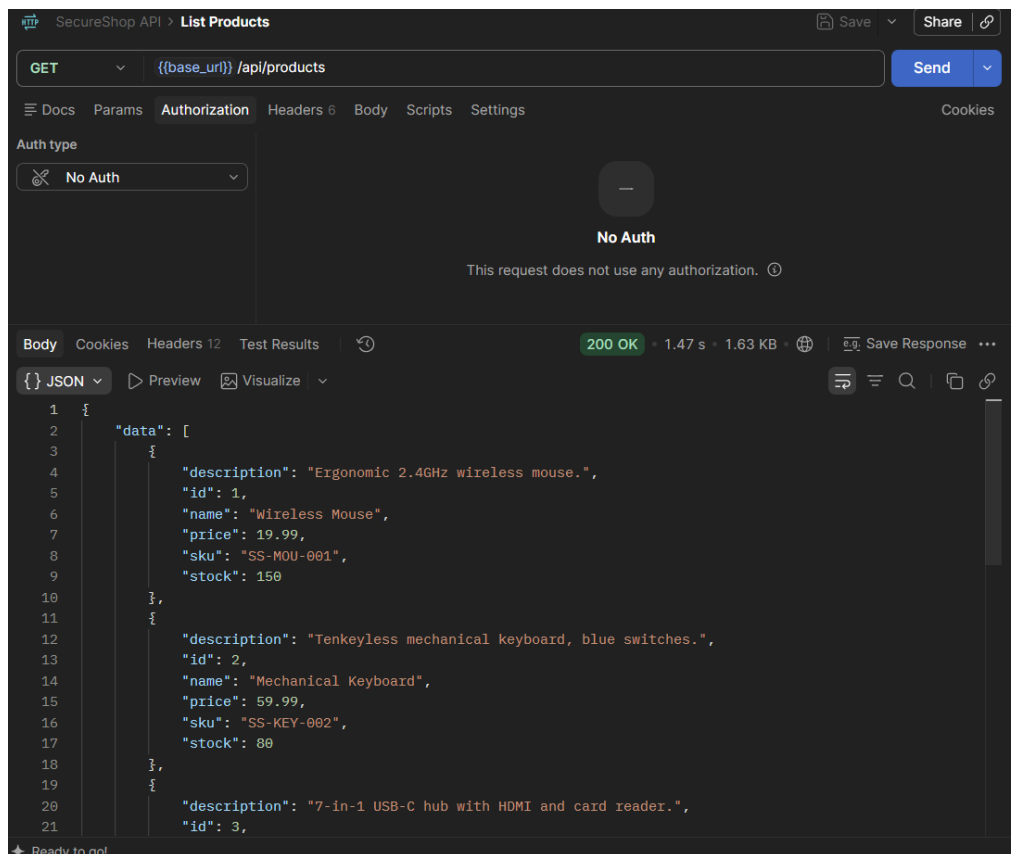
Login User



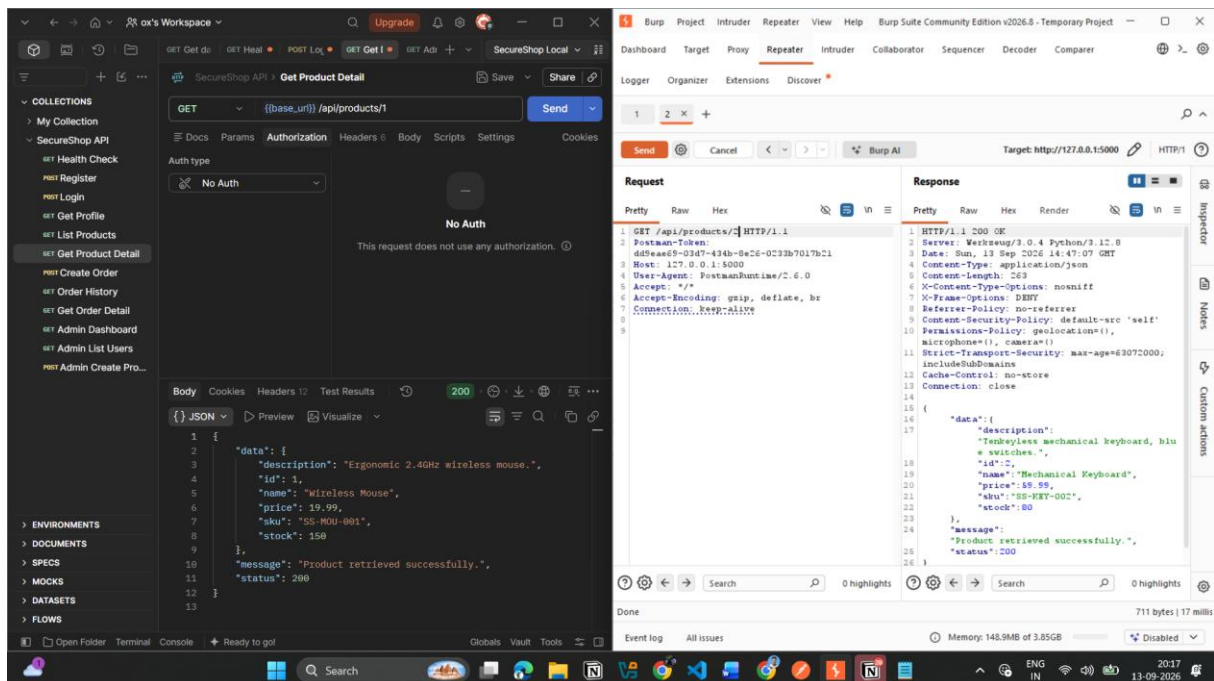
Get Profile



List Products



Products with ID through proxy can intercept in burp and change (or whatever you want)



admin dashboard

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. A GET request to `http://127.0.0.1:5000/api/admin/dashboard` is being repeated. The request headers include `Authorization: Bearer ([jwt_token])`. The response is a 200 OK status with a JSON body containing dashboard statistics.

Request Headers:

Key	Value
Authorization	Bearer ([jwt_token])

Response Body (JSON):

```
{  "data": {    "total_orders": 8,    "total_products": 6,    "total_users": 5  },  "message": "Admin dashboard retrieved successfully.",  "status": 200}
```

Admin list user

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. A GET request to `http://127.0.0.1:5000/api/admin/users` is being repeated. The request is marked as 'No Auth'. The response is a 200 OK status with a JSON body containing a list of users.

Request Headers:

Key	Value
Authorization	No Auth

Response Body (JSON):

```
{  "data": [    {      "address": "1 Fictional Ave, Sampleton",      "created_at": "2026-09-12T15:34:58.754552Z",      "email": "admin@secureshop.local",      "id": 1,      "name": "Ava Administrator",      "phone": "555-0100",      "role": "admin"    },    {      "address": "12 Sample St, Fictionville",      "created_at": "2026-09-12T15:34:58.754552Z",      "email": "admin@secureshop.local",      "id": 2,      "name": "John Doe",      "phone": "555-0101",      "role": "user"    }  ],  "message": "Admin list retrieved successfully.",  "status": 200}
```

Admin create product

The screenshot displays the Burp Suite interface with a successful POST request to the endpoint `/api/admin/products`. The request is configured with the URL `{{base_url}}/api/admin/products` and no authentication. The response status is 201, and the body contains a JSON object indicating a successful product creation.

Request Details:

- Method: POST
- URL: `{{base_url}}/api/admin/products`
- Auth type: No Auth

Response Body (JSON):

```
1 {
2   "data": {
3     "description": "A fictional product for testing.",
4     "id": 7,
5     "name": "Sample Gadget",
6     "price": 15.8,
7     "sku": "SS-TEST-999",
8     "stock": 50
9   },
10  "message": "Product created successfully.",
11  "status": 201
12 }
13
```

The right-hand pane shows the "Intercept is on" message, indicating that Burp Suite is actively intercepting traffic between the browser and the target servers.

Some of the test cases

Registration

Sr.	Use Case	Expected Result
1	Register with valid name, email, password	201, user created with role user
2	Register with an email that's already taken	400, generic error (doesn't confirm email exists)
3	Register with password under 8 characters	400
4	Register with password missing a number or letter	400
5	Register with invalid email format (notanemail)	400
6	Register with empty name field	400
7	Register while passing "role": "admin" in the body	201, but role is still user (never honors client-supplied role)

Login

Sr	Use Case	Expected Result
8	Login with correct email + password	200, returns a JWT
9	Login with correct email, wrong password	401
10	Login with an email that doesn't exist	401, same message as #9
11	Login with empty email/password	400
12	Send 6+ rapid failed logins from same IP	429 after the threshold

Authentication

#	Use Case	Expected Result
13	Access /api/profile with no token	401
14	Access /api/profile with a malformed token (Bearer xyz)	401
15	Access /api/profile with a tampered/re-signed token	401
16	Access /api/profile with an expired token	401
17	Access /api/profile with a valid token	200, returns own profile only

Authorization / RBAC

#	Use Case	Expected Result
18	Normal user calls GET /api/admin/dashboard	403
19	Normal user calls GET /api/admin/users	403
20	Normal user calls POST /api/admin/products	403
21	Admin calls all three admin routes	200 on each

Object-Level Authorization (IDOR)

Sr	Use Case	Expected Result
22	User A fetches their own order via GET /api/orders/<id>	200, correct order
23	User A fetches User B's order ID via GET /api/orders/<id>	404 (not exposed)
24	User A enumerates sequential order IDs to find others' orders	All non-owned IDs return 404

Products

Sr	Use Case	Expected Result
25	List all products (no auth)	200, array of 6 products
26	Get single product by valid ID	200
27	Get product by non-existent ID (e.g. 9999)	404
28	Get product by non-numeric ID (abc)	400
29	DELETE /api/products (unauthorized method)	405

Orders

Sr	Use Case	Expected Result
30	Create order with valid product_id + quantity	201, ossssorder created, stock decremented
31	Create order with quantity greater than available stock	400
32	Create order with quantity: 0 or negative	400
33	Create order with quantity: "two" (wrong type)	400
34	Create order with non-existent product_id	404
35	Create order with no auth token	401
36	View order history as User A	200, only User A's orders returned

Admin

Sr	Use Case	Expected Result
37	Admin dashboard returns aggregate counts	200, matches actual DB counts
38	Admin lists all users	200, full admin view (includes phone/address)
39	Admin creates a new product with valid data	201
40	Admin creates product with duplicate SKU	400
41	Admin creates product with negative price/stock	400

Cross-Cutting Security Checks

Sr	Use Case	Expected Result
42	Inspect response headers on any endpoint	X-Content-Type-Options, X-Frame-Options, CSP, Cache-Control: no-store present
43	Trigger a malformed-JSON request body	Clean 400, no stack trace
44	Compare /api/profile fields vs /api/admin/users fields	Profile never leaks phone/address/hash
45	Inspect database/secureshop.db directly	Passwords are hashed, never plaintext
46	Perform a mix of logins/admin actions, then check reports/audit.log	Events logged, no passwords/tokens present

Once every case above behaves as expected on the secure API, flip TRAINING_MODE=true and repeat the **same 46 use cases** against /api/training/* — some will now fail (by design), and identifying *which ones* and *why* is the actual exercise.

Activate and use Training Mode:

- Open .env, set **TRAINING_MODE=true**, save
- Run python scripts\init_database.py — randomly selects which vulnerabilities are active
- Restart: python app.py — console shows a warning banner
- Verify: Invoke-WebRequest http://127.0.0.1:5000/api/health → should show "training_mode": true

What changes:

- New routes unlock under /api/training/* (orders, admin/users, profile, login, products)
- Each mirrors a real secure endpoint but may skip a check (ownership, role, rate limit, validation)
- Every response gets an X-Training-Mode: enabled header as a reminder

To test:

- Run your 46 test cases against /api/training/* and note where behavior differs from the secure API
- Don't open docs/developer-remediation-guide.md until you're done — it has the answers

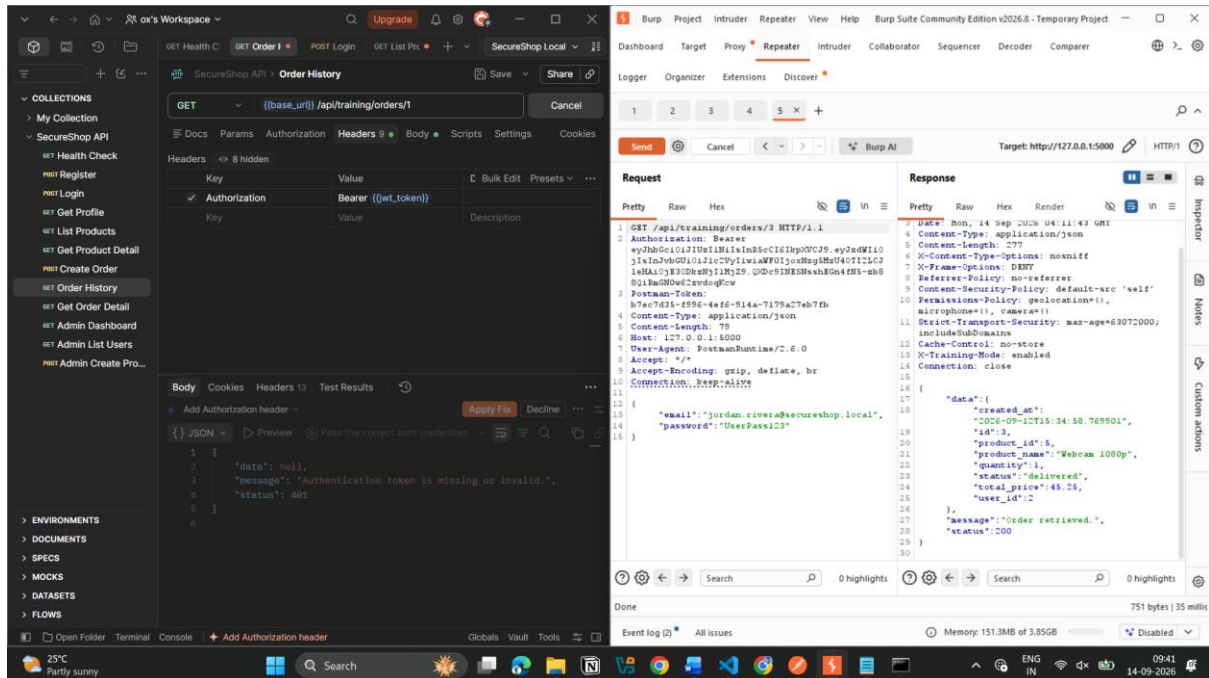
To disable:

- Set TRAINING_MODE=false in .env, restart — training routes become unreachable entirely

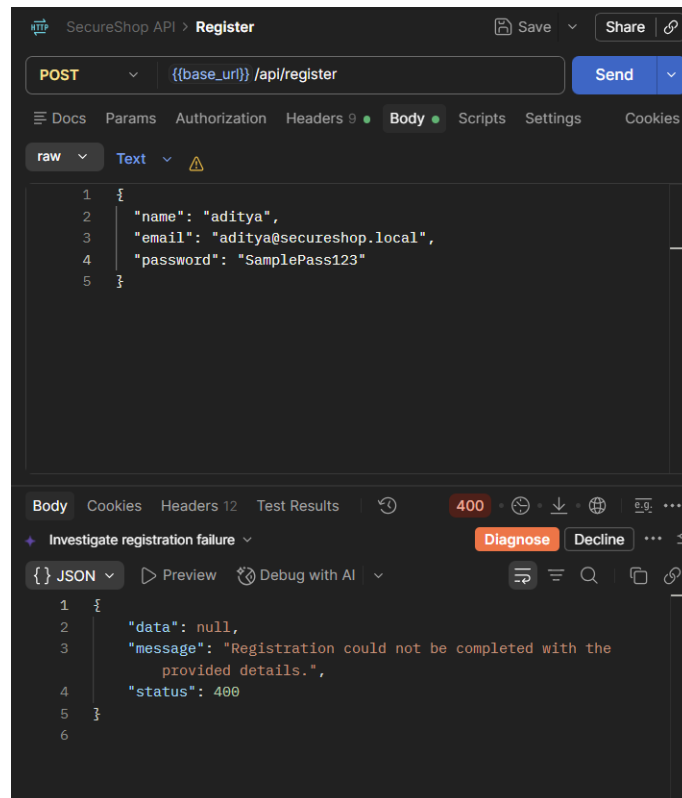
@AdityaBhosale

Dumb screenshots:

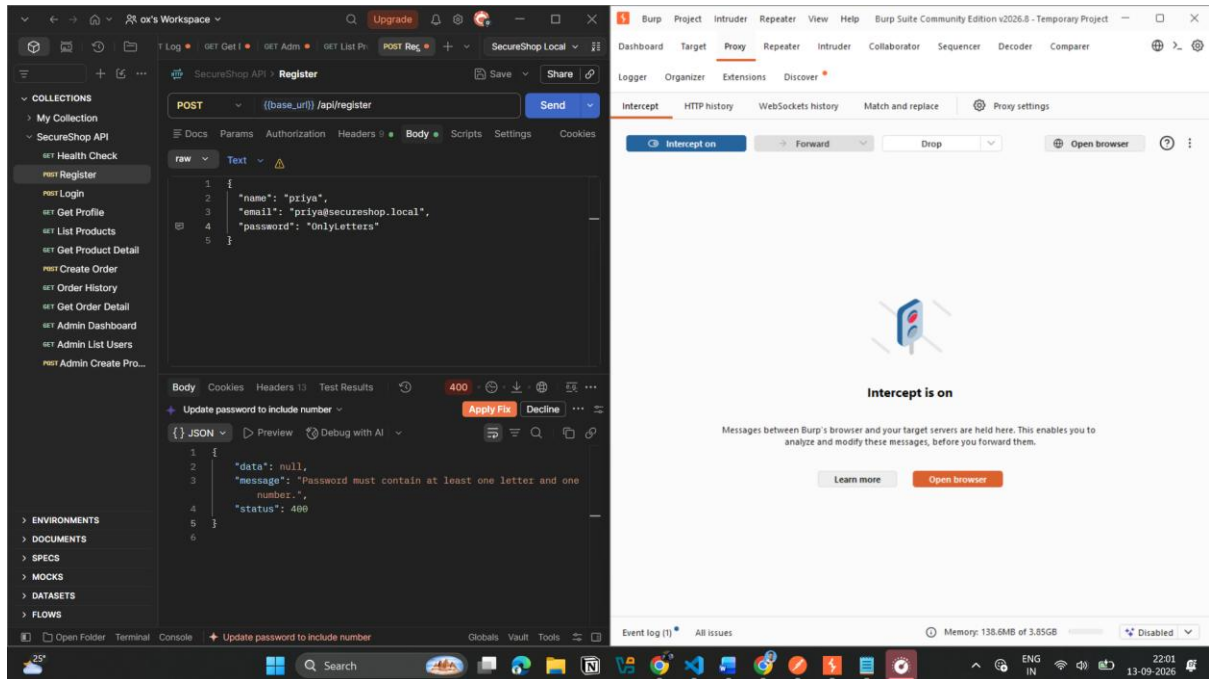
For training mode use /api/training/* endpoint to test



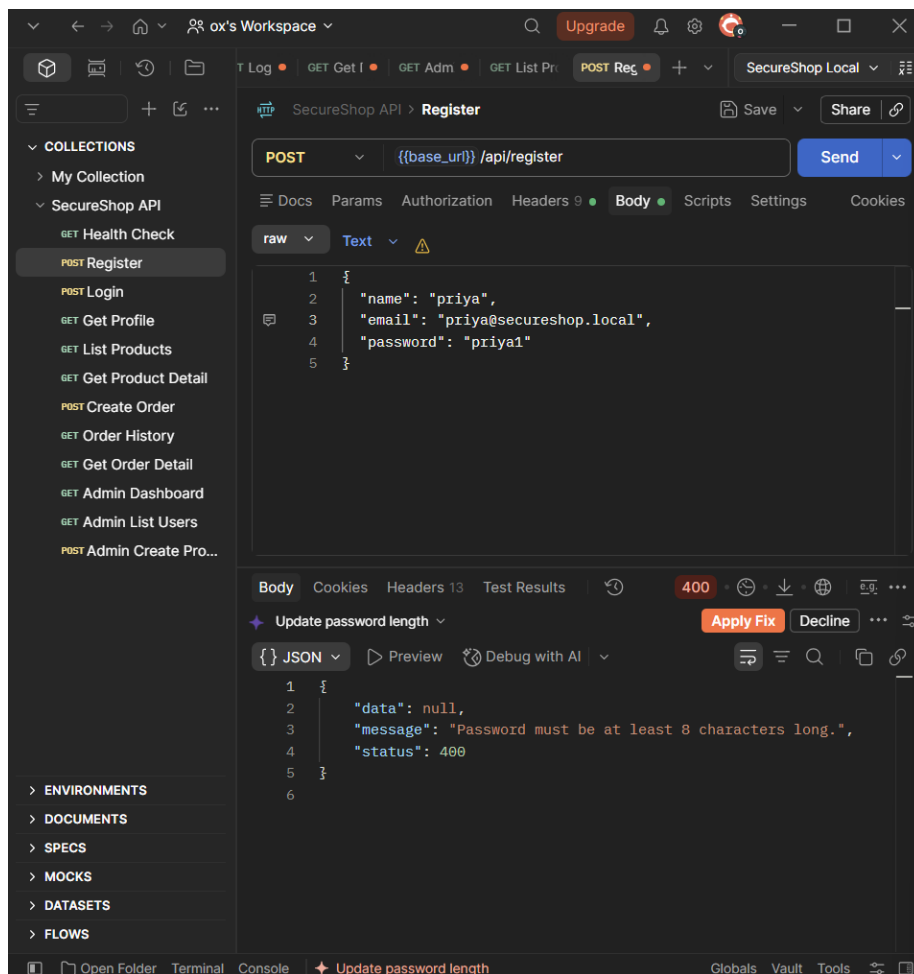
Trying to registered with existing mail



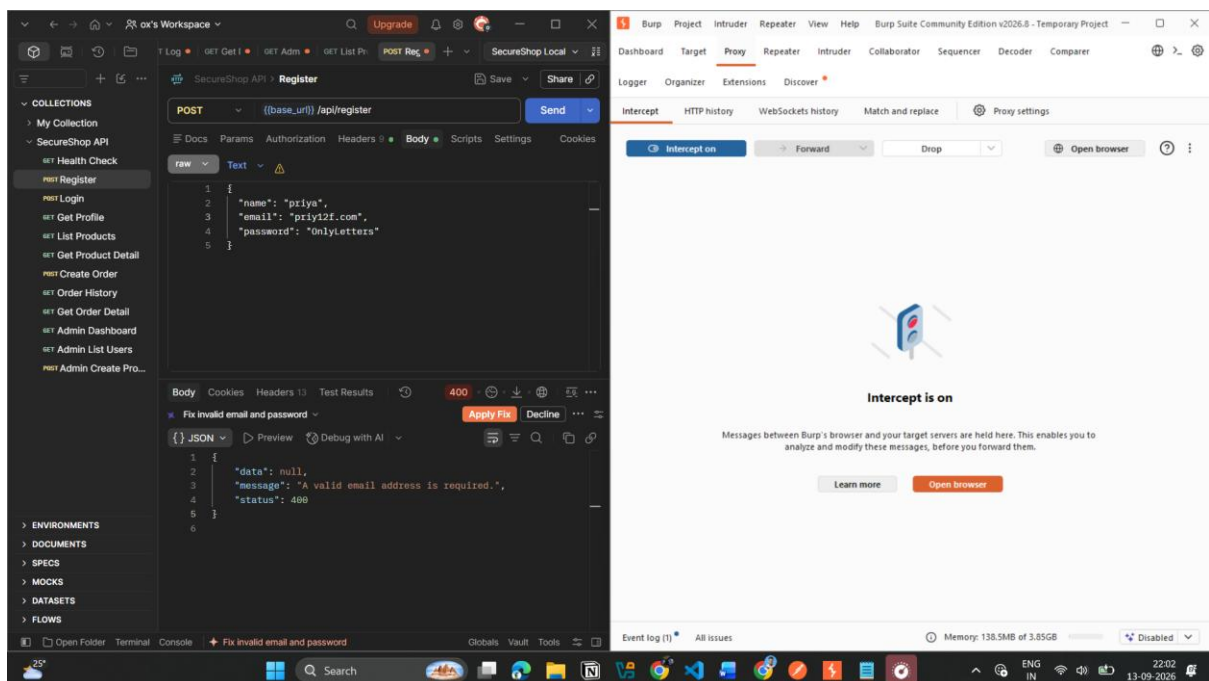
Reg with only letters



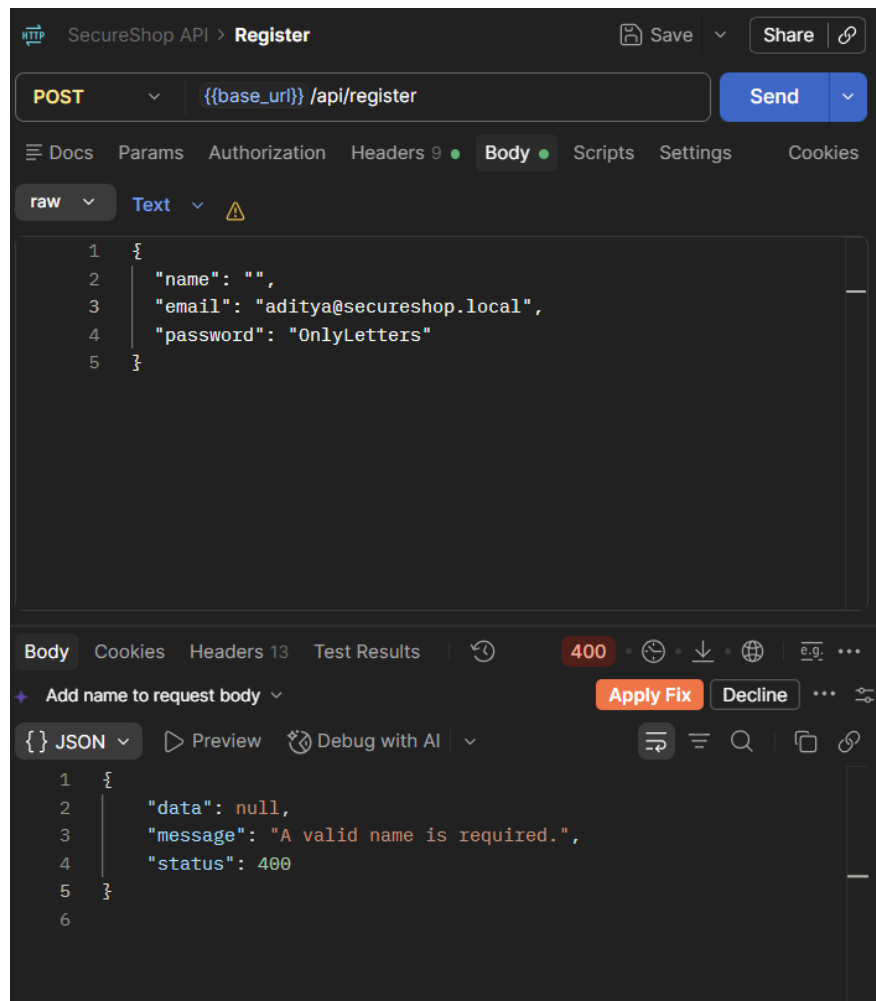
Registering with less than 8-character password



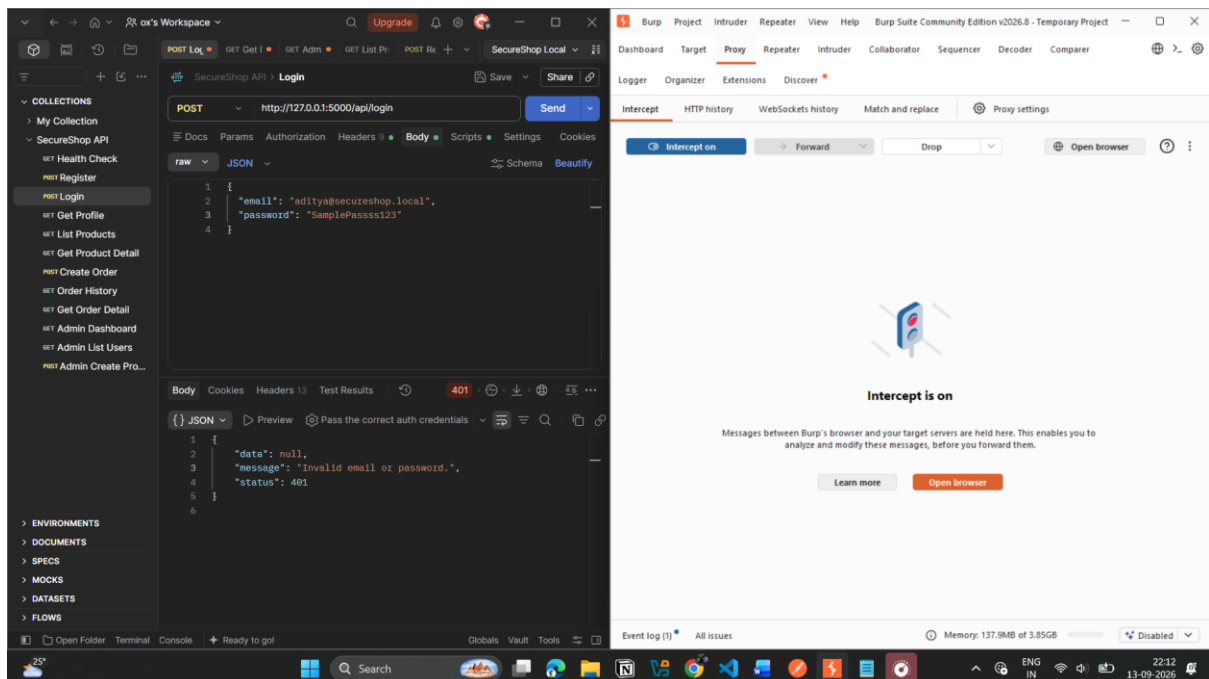
Invalid email format



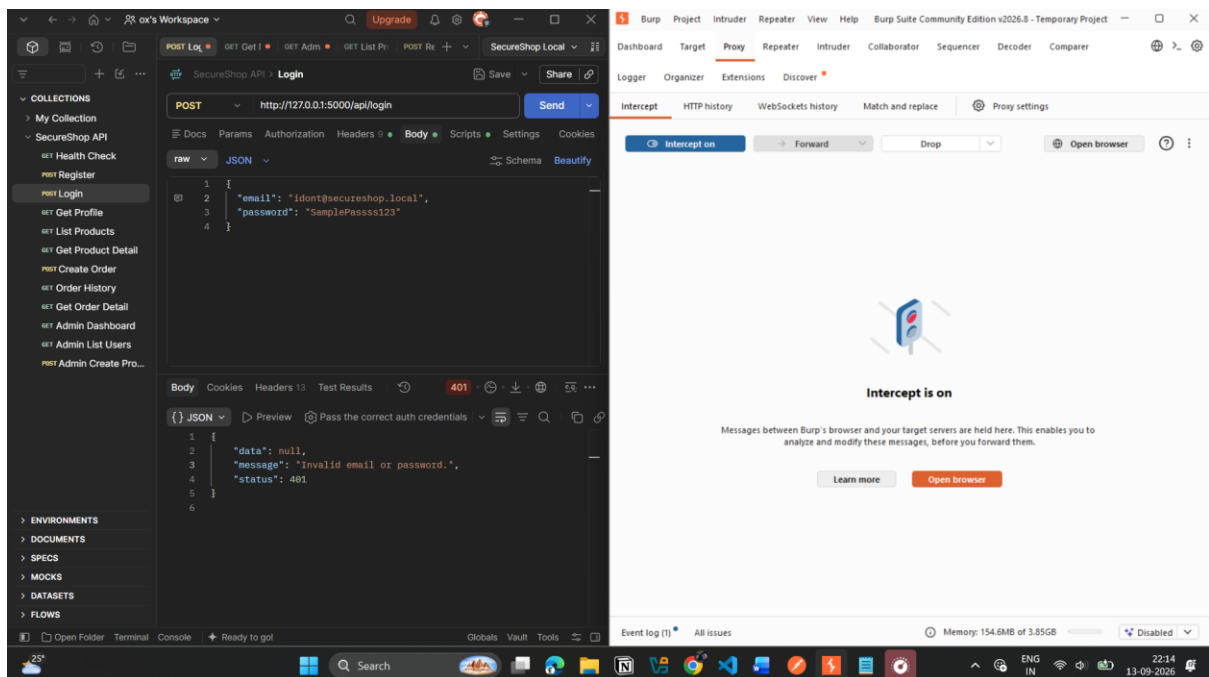
Reg without name



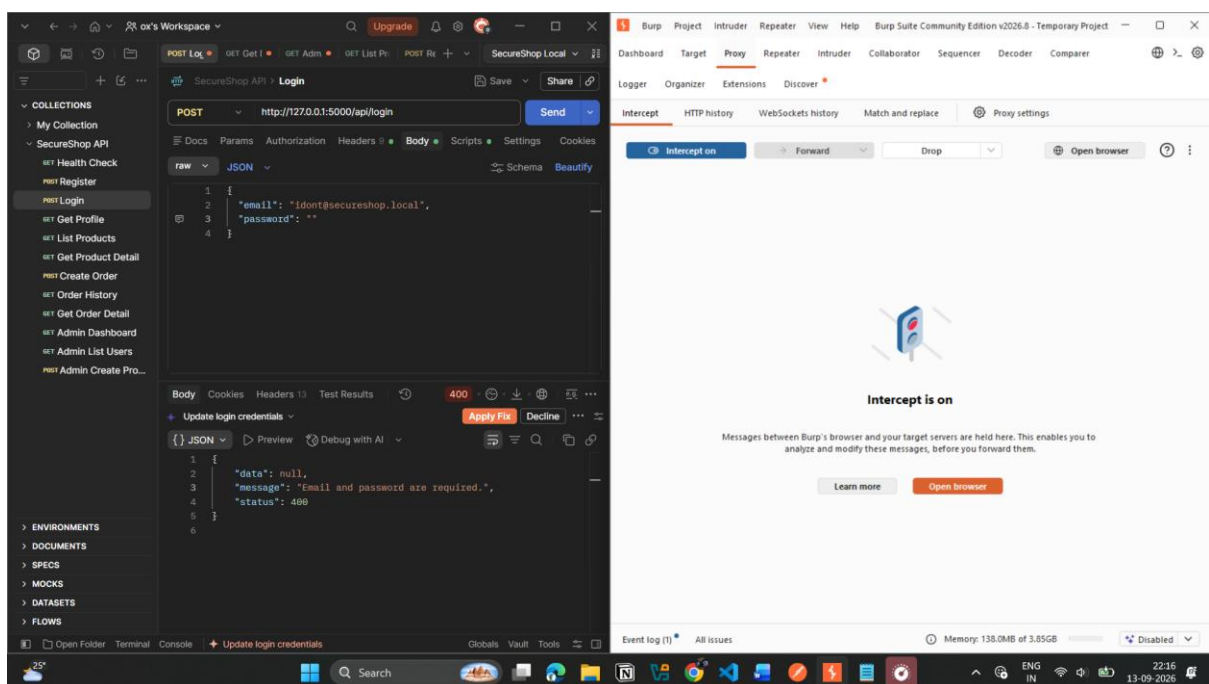
Wrong email or password



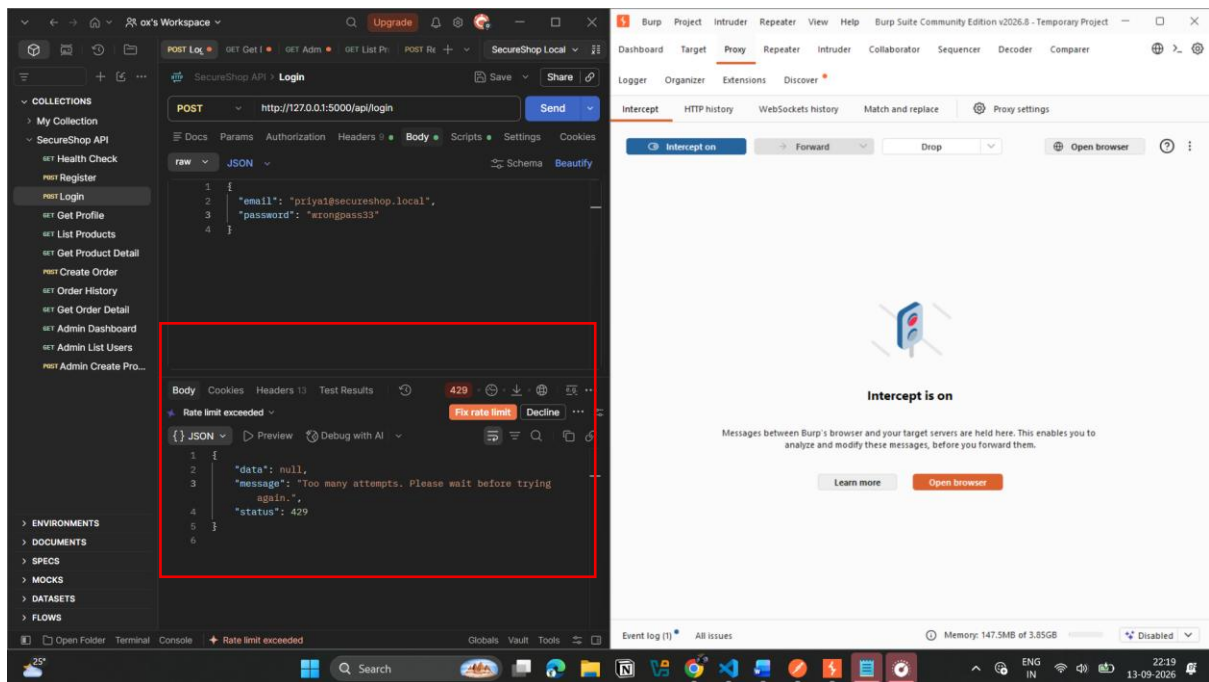
Trying login with email that does not exist



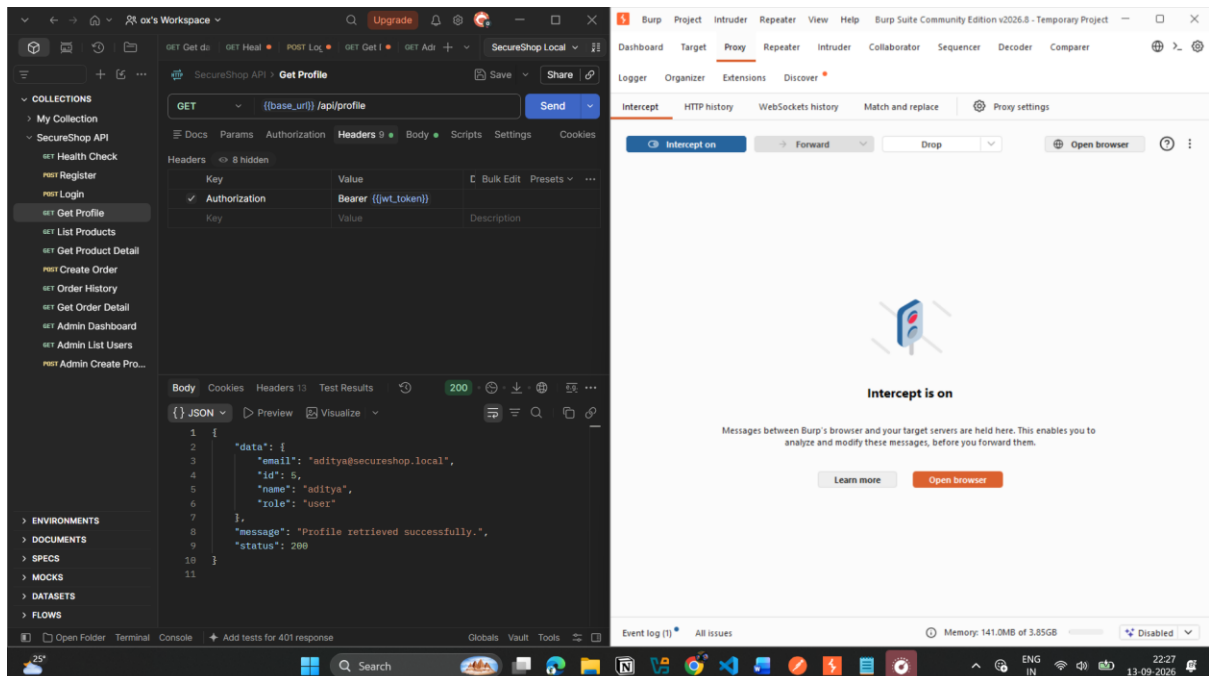
Empty email or password



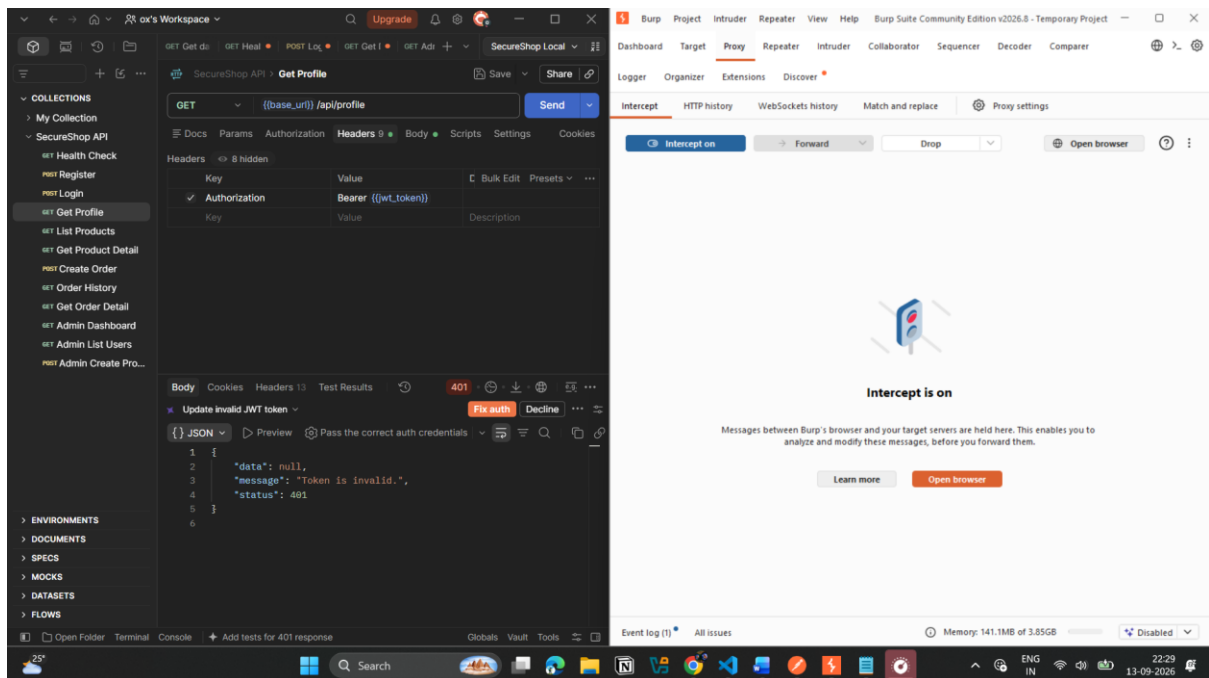
More than 6 failed attempts



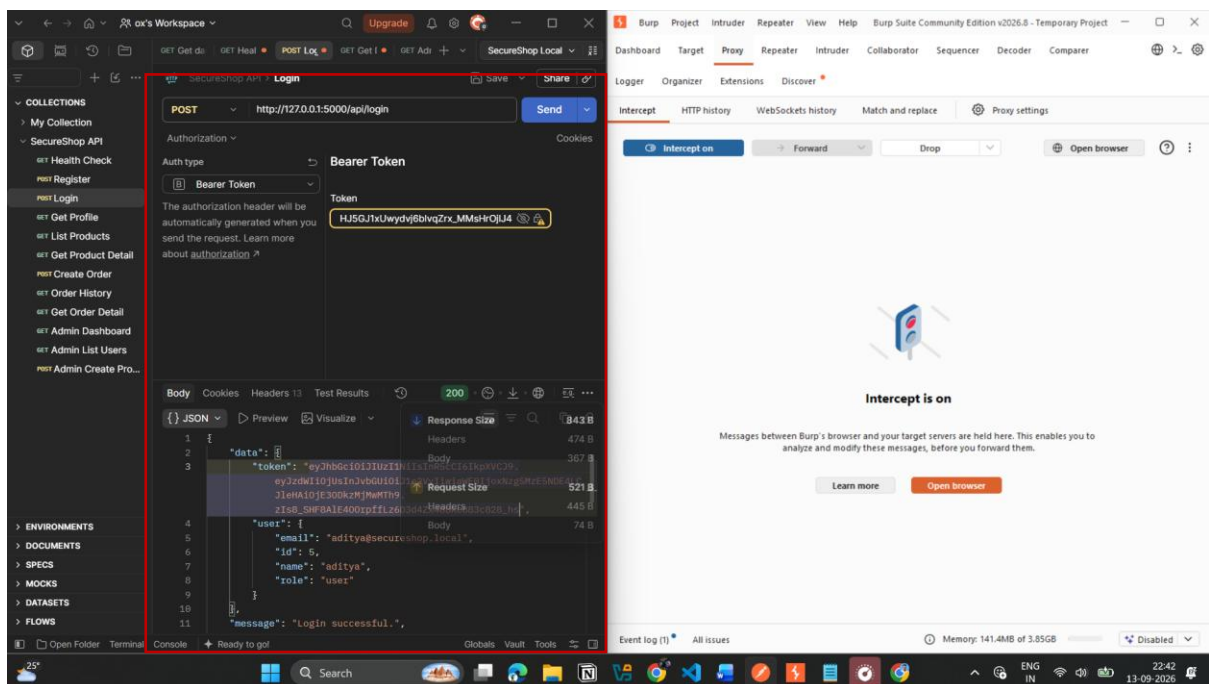
Getting profile with valid token



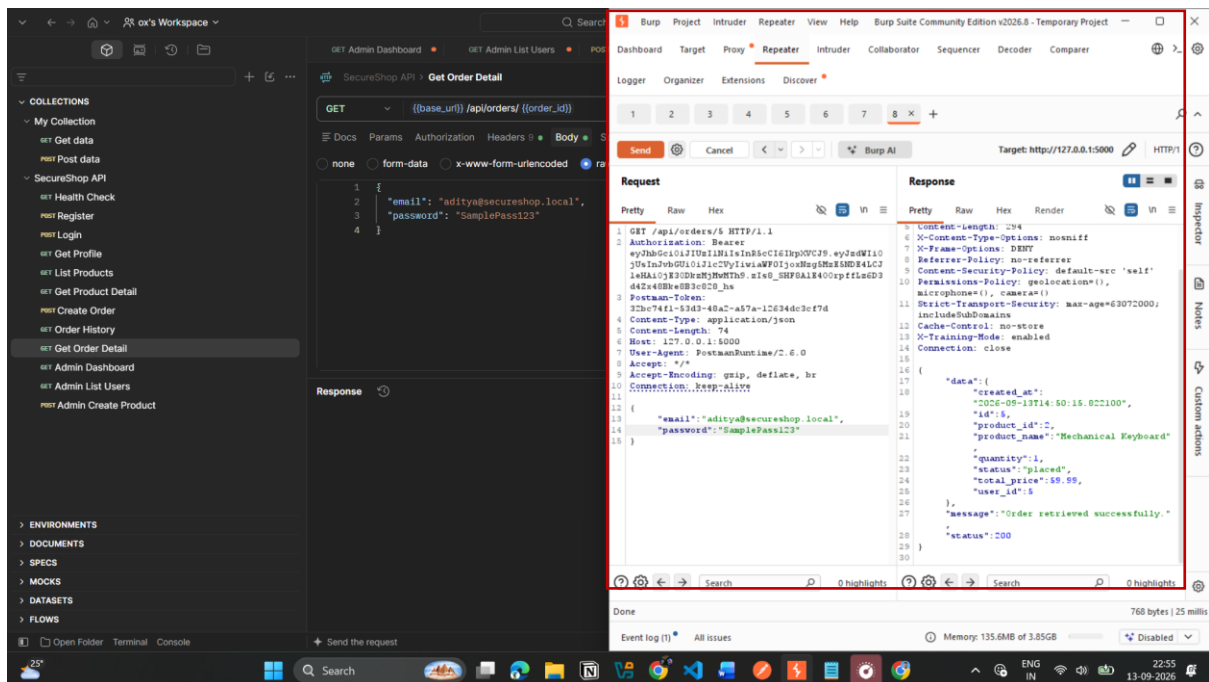
trying to access with modified tokens



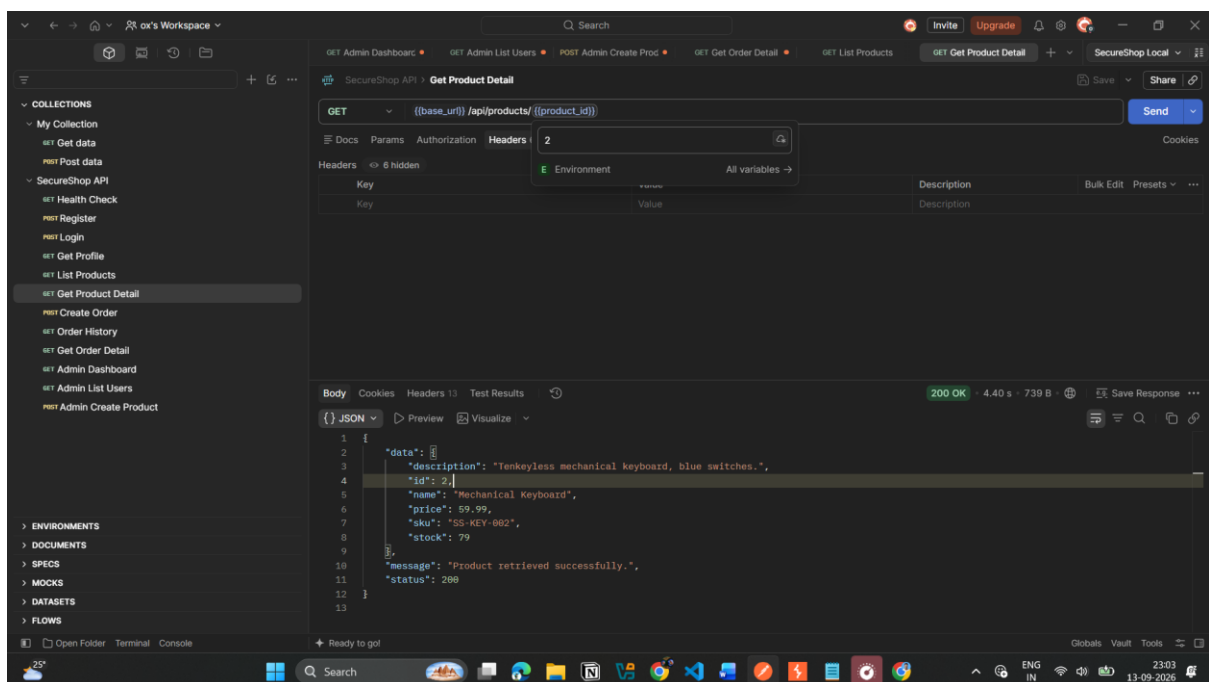
Can sing in with expired token (with training mode)



Can see other Order ID details via repeater



Get single product with ID



Ordering more than stock

The screenshot shows the Burp Suite interface with a POST request to `/api/orders` in the Repeater tab. The request body is a JSON object: `{ "product_id": 2, "quantity": 80 }`. The response is a 400 Bad Request with a message: `"Insufficient stock for the requested quantity."` and status 400.

```
POST /api/orders HTTP/1.1
Host: 127.0.0.1:5000
Content-Type: application/json
Authorization: Bearer ((jwt_token))
{
  "product_id": 2,
  "quantity": 80
}
```

```
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.0.4 Python/3.12.8
Date: Mon, 14 Sep 2026 02:37:39 GMT
Content-Type: application/json
Content-Length: 99
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Content-Security-Policy: default-src 'self'
Permissions-Policy: geolocation=(), microphone=(), camera=()
Strict-Transport-Security: max-age=6307200; includeSubDomains
Cache-Control: no-store
X-Training-Mode: enabled
Connection: close

{"data": null, "message": "Insufficient stock for the requested quantity.", "status": 400}
```

adding product with negative price or stock

The screenshot shows the Burp Suite interface with a POST request to `/api/admin/products` in the Repeater tab. The request body is a JSON object: `{ "name": "a", "description": "a", "price": -15.00, "stock": 100, "sku": "SS-TEST-123456" }`. The response is a 400 Bad Request with a message: `"Price must be greater than zero."` and status 400.

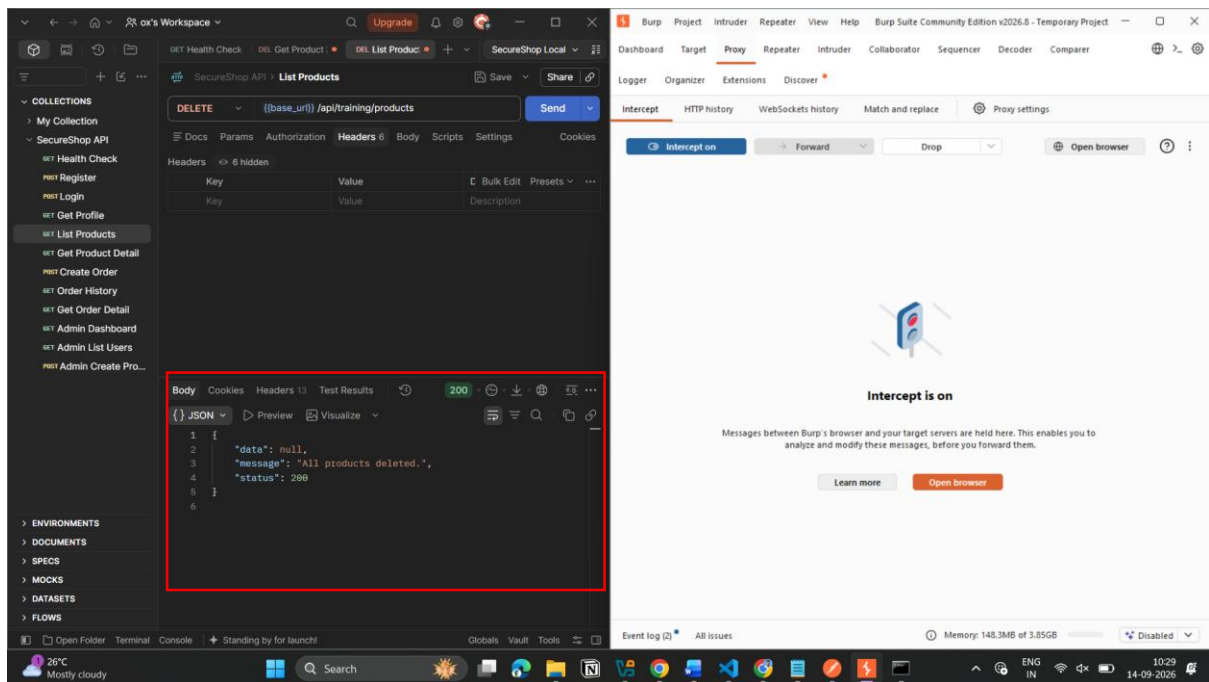
```
POST /api/admin/products HTTP/1.1
Host: 127.0.0.1:5000
Content-Type: application/json
{
  "name": "a",
  "description": "a",
  "price": -15.00,
  "stock": 100,
  "sku": "SS-TEST-123456"
}
```

```
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.0.4 Python/3.12.8
Date: Mon, 14 Sep 2026 02:37:39 GMT
Content-Type: application/json
Content-Length: 99
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Content-Security-Policy: default-src 'self'
Permissions-Policy: geolocation=(), microphone=(), camera=()
Strict-Transport-Security: max-age=6307200; includeSubDomains
Cache-Control: no-store
X-Training-Mode: enabled
Connection: close

{"data": null, "message": "Price must be greater than zero.", "status": 400}
```

@AdityaBhosale

Normal user can delete products BOLA



Grateful to the open-source community and happy to contribute even in a small way. If you have any questions or feedback, feel free to email me at adityabhosale212223@gmail.com

Thank you